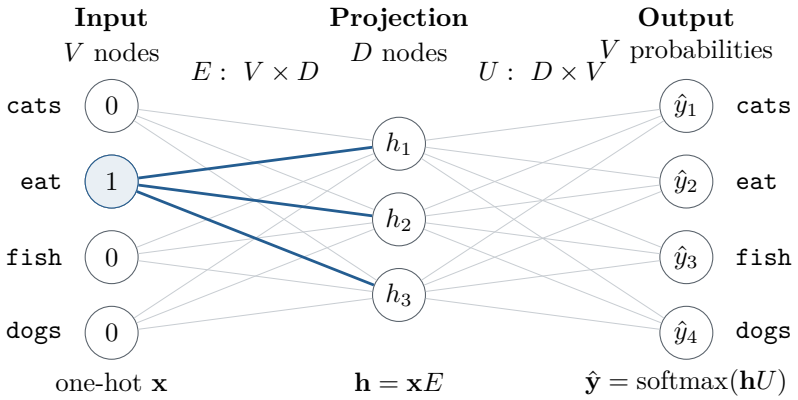
Word2Vec: Structure and Questions

A short guide based on our discussion.

Scope. The main example uses Skip-gram with full Softmax: one center word predicts nearby context words.



Example: $V = 4$, $D = 3$, input = `eat`. Blue connections form the input word vector for `eat`.

Object	Size	Meaning
Input $\mathbf{x}$	$1 \times V$	One-hot vector identifying the center word.
Weights E	$V \times D$	Each row is one word's input embedding.
Hidden $\mathbf{h}$	$1 \times D$	The selected embedding; one coordinate per node.
Weights U	$D \times V$	Each column is one word's output embedding.
Output $\hat{\mathbf{y}}$	$1 \times V$	Probability of each word as a context word.

Why the weights are the word vectors. Rows of E follow the vocabulary order. For illustration:

$$E = \begin{array}{c|ccc} \texttt{cats} & 0.1 & 0.3 & -0.2 \\ \texttt{eat} & 0.2 & -0.1 & 0.4 \\ \texttt{fish} & -0.3 & 0.2 & 0.1 \\ \texttt{dogs} & 0.4 & 0.1 & -0.1 \end{array}$$

$$\underbrace{[0,1,0,0]}_{\text{input: eat}} E = 0E_{\texttt{cats},:} + 1E_{\texttt{eat},:} + 0E_{\texttt{fish},:} + 0E_{\texttt{dogs},:} = [0.2, -0.1, 0.4].$$

One-hot multiplication selects one row. These three stored weights are the input embedding of `eat`; the hidden activations equal that embedding. This projection adds no bias or nonlinear activation. Both E and U are trained; E is commonly kept as the learned word vectors.

Questions and short answers

Q1. Is Word2Vec an embedding algorithm?

Yes. It learns static word embeddings. Skip-gram predicts context words from a center word. CBOW predicts a center word from surrounding words.

Q2. Does the input layer have one node per vocabulary word?

Yes, in the one-hot view: its size is V. In code, a word ID can directly select a row of E, without creating a long one-hot vector.

Q3. Does one-hot represent the entire input sentence?

Here it identifies one center word. A sentence supplies many center–context training pairs.

Q4. What determines the number of hidden nodes?

We choose the embedding dimension D before training. It is a hyperparameter, independent of vocabulary size. A larger D uses more parameters and may need more data; it does not guarantee better results.

Q5. How does a word correspond to a row of weights?

A fixed word-to-ID map assigns its row in E. Training changes the numbers in that row. For one input word, its D outgoing weights form its input vector.

Q6. What does the output layer predict?

It outputs V probabilities: $\hat{y}_j = P(\text{context word } j \mid \text{center word})$. They sum to 1. The hidden layer holds the word vector; the output layer provides the prediction.

Q7. Are there only two outputs, one for the previous and one for the next word?

There are V output nodes. A left/right window of size 1 can provide two training pairs. Both pairs use the same output layer and weights.

Q8. Is the output also one-hot?

The prediction is a probability vector, such as $[0.45, 0.10, 0.45]$. A single training pair has a one-hot target, such as $[1, 0, 0]$. The target can also be stored as a word ID.

Q9. Does Skip-gram predict the next word?

Its targets include words on both sides within the chosen window. Basic Skip-gram uses the same distribution for these positions. Predicting only the next word is a different training objective.

Examples, boundaries, and training

Q10. For `cats eat fish`, what are the inputs, targets, and boundaries?

Use vocabulary [`cats`, `eat`, `fish`] and a window of one word on each side. The complete set of training pairs is:

Center	Input $\mathbf{x}$	Context target	Target $\mathbf{y}$
`cats`	$[1, 0, 0]$	`eat`	$[0, 1, 0]$
`eat`	$[0, 1, 0]$	`cats`	$[1, 0, 0]$
`eat`	$[0, 1, 0]$	`fish`	$[0, 0, 1]$
`fish`	$[0, 0, 1]$	`eat`	$[0, 1, 0]$

At sentence boundaries, use only neighbors that exist. Here, `cats` and `fish` each provide one pair; `eat` provides two. No padding is needed. For `eat`, the ideal output on this toy data approaches $[0.5, 0, 0.5]$. An actual finite Softmax output has strictly positive entries.

Q11. What if the data also contains `dogs eat meat`?

Expand the vocabulary to [`cats`, `eat`, `fish`, `dogs`, `meat`]. For center `eat`, the four targets are `cats`, `fish`, `dogs`, and `meat`.

If each sentence occurs once, the ideal distribution for `eat` is

$$[1/4,\ 0,\ 1/4,\ 1/4,\ 1/4].$$

If `cats eat fish` occurs three times and `dogs eat meat` once, it is

$$[3/8,\ 0,\ 3/8,\ 1/8,\ 1/8].$$

These are idealized frequencies for equally weighted pairs with the fixed window. The input `eat` gives the same distribution in both sentences: this model receives only the center word.

Q12. Are the training targets obtained from statistics over large text data?

We choose the task and window; the text supplies each pair's correct answer. Repeated training learns the context statistics. We do not need to precompute a complete probability table.

For N pairs (w_k, c_k), minimize the average cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{k=1}^{N} \log P(c_k \mid w_k).$$

Backpropagation computes gradients. An optimizer updates E and U. The vectors typically start from small random values and learn through these updates.

Sources. Mikolov et al., Word2Vec architectures (2013); Mikolov et al., Skip-gram formulation (2013); PyTorch, Word Embeddings tutorial.